Data Platform Engineer - AWS DevOps - 60-minute interview

Interviewer guide. Keep the answer guidance separate from the candidate handout. Use the candidate exercise during the practical task. Use the separate example answers and level calibration to distinguish regular/mid expectations from optional stronger answers. Prepared on 16 October 2026 from the job advertisement and three supplied CV screenshots.

Conduct the entire interview in English, including the introduction, technical questions, exercise discussion, follow-ups, and closing. Assess communication throughout; there is no separate English-language test or language-switching block.

1. Purpose and role alignment

The target role is **Data Platform Engineer - AWS DevOps**, as clarified by the interviewer. Assess whether the candidate can build, deploy, secure, and operate an AWS data platform used by data engineers and data consumers. Python supports automation; the outcome is a reliable, observable, repeatably deployable platform.

Responsibility	Evidence to look for
AWS data platform	Understands ingestion, S3 storage, processing, cataloguing, and querying
IaC and platform standards	Builds reusable modules, separates environments, manages state and changes
CI/CD	Deploys infrastructure and job artifacts with tests and promotion controls
Security and networking	Understands IAM, KMS, cross-account access, and private-subnet connectivity
Data operations	Measures freshness and completeness; handles incidents, replay, and backfill
Supporting data engineers	Provides deployment patterns, monitoring, and operational instructions

Do not require complex Spark transformations in this hour. Ask how processing jobs are deployed, triggered, secured, and monitored. Accept a different architecture when the candidate can explain how it meets the requirements.

The advertisement calls for relevant experience, cloud work, programming, IaC, and communication skills. Containers and specific data platforms are additional skills; expand those topics when they matter to the actual project. Capgemini job advertisement.

What to validate from this CV

CV claim	What to verify
End-to-end cross-account S3 migration	Personal contribution, completeness checks, consumer cutover, failure handling
Python, boto3, pytest, structured logging	A short verbal code review and a concrete automation/testing example
Terraform extensions and CloudFormation	State, plans, drift, and partially failed deployments
GitHub Actions, OIDC, post-deployment tests	Authentication flow, permission boundaries, and useful integration tests
IAM access-key rotation through Lambda and Secrets Manager	Partial failures, retries, secret consumers, and recovery
GxP-regulated platform and change management	Concrete review, testing, approval, and audit evidence
Financial operations and team leadership	Clear communication, troubleshooting, and the scope of earlier technical work

The CV lists a Cloud Engineer role starting in July 2025. The advertisement asks for at least three years in a similar role. Clarify earlier technical responsibilities and confirm the formal experience requirement with the recruiter separately. Dates and job titles alone do not settle technical competence. Certifications and time in financial operations do not replace engineering evidence.

Do not infer missing skills solely from three screenshots. Record confirmed examples and unassessed areas separately.

2. Agenda - exactly 60 minutes, entirely in English

Minutes	Block	Main objective
0-3	Introduction	Explain the AWS data-platform role and interview format
3-7	Q1: project ownership	Establish what the candidate personally delivered and operated
7-17	Q2: AWS platform and security	Discuss data flow, access, processing, and replay
17-27	Q3: Terraform	Discuss state, environments, and partial deployment failure
27-34	Q4: CI/CD and OIDC	Deploy infrastructure and data-processing artifacts safely
34-44	Q5: Python	Review two short functions verbally and discuss real automation experience
44-55	Q6: operations and networking	Diagnose an incident, recover data, and communicate impact
55-60	Candidate questions	Discuss the project, responsibilities, and next steps

Ask each main question and select follow-ups according to the answer and time available. The follow-up lists are a question bank, not a checklist to read out. Protect the final five minutes for the candidate's questions.

Opening script

> Thanks for joining. We have 60 minutes, and we will conduct the whole interview in English. We will discuss your AWS projects, platform design, Terraform, deployment pipelines, and production troubleshooting. There will also be a short verbal Python code review, with no live-coding requirement. I am interested in what you personally did, why you made particular decisions, and how you verified the outcome. Approximate, anonymised examples are absolutely fine. You can use API documentation during the exercise. We will leave the last five minutes for your questions.

Useful prompts throughout:

- "Could you walk me through a specific example?"
- "Which part did you implement yourself?"
- "What evidence would confirm that hypothesis?"
- "What would happen if that operation failed halfway through?"
- "Could you explain that to a data engineer or business stakeholder?"

3. Questions and interviewer guidance

Q1 - Ownership of an AWS project, 4 minutes

Ask:

> Your CV mentions an end-to-end cross-account S3 migration. Could you sketch the data flow, explain who consumed the data, and identify the components you personally designed, implemented, or operated after deployment?

Choose a follow-up:

- Roughly how many accounts, objects, and how much data were involved?
- What already existed when you joined, and what did other team members deliver?
- Describe one decision you made and an alternative you rejected.
- What was the most difficult issue, and how did you confirm its cause?
- What technical automation work did you do before your current role, and who used it?

Strong evidence: separates personal and team contributions; explains the consumers and flow; gives a concrete change, test, and result; acknowledges areas outside their responsibility.

If the answer remains a list of services or team activities, ask for one concrete example. Using "we" is not, by itself, a negative signal.

Q2 - AWS data platform and security, 10 minutes

Main scenario - around 6 minutes:

> A supplier delivers CSV and JSON files to S3. Data engineers need to process them, and analysts need to query the prepared data. We have separate dev, test, and production environments. New data must be ready for consumption within 15 minutes of arrival. How would you provision and operate this AWS platform? Start by asking about missing requirements, then sketch the flow.

This is an interview scenario, not a confirmed Capgemini architecture. The 15-minute target is an exercise assumption. Do not judge capacity choices without providing data volumes and transformation requirements.

Choose two follow-ups:

- Where would you keep unchanged source data and prepared datasets?
- What triggers processing, and how would you handle buffering and dependencies?

- How would you choose between Lambda and Glue for this workload?
- What happens if the supplier sends the same file again or changes its schema?
- How would you backfill one day without double-counting its data?
- What reusable module or deployment pattern would you give a data engineer so that each new job does not require manual infrastructure configuration?

Strong evidence: asks about volume, frequency, consumers, security, and cost; separates source data from outputs; explains processing choices and triggers; considers schema, validation, quarantining invalid data, execution history, and safe retries. Can identify which resources belong in IaC and what to monitor.

One possible discussion path is S3 raw → triggering and optional event buffering → processing → S3 curated → catalogue and queries. Glue can provide processing and a Data Catalog; Athena is one possible query layer. Do not require every service or treat this combination as the only valid answer. AWS: Glue and related services.

Security - around 3 minutes:

> The processing job runs in the platform account, while the analyst uses another AWS account. The data is encrypted with a KMS key. How would you restrict the analyst to an approved dataset, and how would you investigate AccessDenied when they try to read it?

Look for distinct execution and consumer roles, IAM policies, S3 resource policies, and KMS permissions and key policies. The candidate should identify the actual identity, operation, account, and resource. If they propose Lake Formation, ask about its additional permissions layer; do not make it an unstated prerequisite. AWS: access and encryption troubleshooting.

Final minute:

> What could drive the largest costs in your design, and how would you check that an optimisation does not compromise data delivery time?

Look for costs tied to the proposed workload, measurement, and trade-offs against the delivery target. Do not ask for service prices from memory. Naming services alone does not demonstrate understanding of the platform.

Q3 - Terraform in a team, 10 minutes

Ask:

> You are extending a data-platform Terraform module with raw and curated buckets, IAM roles, a processing job, and alarms for dev, test, and production. Walk me through the change from pull request to production. Where is the state stored, and how do you prevent two people from making conflicting changes?

Required follow-up:

> An apply creates the bucket but fails while configuring its policy. What would you check before the next run? Does reverting the commit revert the infrastructure?

Choose one additional follow-up if time allows:

- How would you separate environment state and configuration and limit the impact of a mistake?
- Someone changes a resource in the console. How would you detect drift and decide which configuration to keep?
- The plan includes deleting a resource that contains data. What would you investigate before proceeding?
- What can be stored in state? Does marking a value `sensitive = true` remove it from state?

Expected reasoning: understands state and the limits of a plan; discusses a controlled remote backend, access restrictions, state protection, destructive changes, and environment separation. After a partial failure, checks actual resources and Terraform state, investigates the error, fixes its cause, and reviews a fresh plan. Terraform does not provide a transactional rollback of an entire apply; a Git revert alone does not perform changes in AWS. Sensitive values can remain in state despite being marked sensitive. HashiCorp: sensitive data.

Distinguish state locking from serialising an entire deployment workflow. The S3 backend supports opt-in locking through `use_lockfile`; DynamoDB-based locking is marked deprecated. An accurate explanation of an older DynamoDB-based setup is valid evidence of experience. HashiCorp: S3 backend.

Probe answers that rely on deleting state, forcing an unlock without checking its owner, or assuming another apply recreates everything from scratch. Ask for specific steps before drawing a conclusion.

Q4 - CI/CD, OIDC, and deployment tests, 7 minutes

Ask:

> Your CV mentions OIDC and post-deployment pytest checks. Describe the pipeline from a pull request to production. How do you deploy Terraform and a versioned Lambda script or Glue job, what do you test, who approves the change, and how does the CI job obtain access to AWS?

Choose follow-ups:

- What restricts role assumption to the intended repository and deployment context?
- How does the role's trust policy differ from the permissions granted to that role?
- What test would detect that resources exist but the intended consumer still cannot read the data?
- How would you test a new job version against representative data and the expected schema before promotion?
- What would you do if apply succeeded but the integration test failed?

Expected reasoning: short-lived credentials through OIDC/STS, permission to request a token in the job, and a restricted trust relationship. Claims such as `aud` and `sub`, plus production environment restrictions, matter. Do not require memorising an exact `sub` format: it depends on the repository and token setup. `id-token: write` alone does not grant access to AWS resources. GitHub: OIDC in AWS.

Look for a distinction between mocks and integration checks: which identity runs the test, whether it represents the consumer, what test resources it creates, how it cleans up, and whether it checks access denial. Infrastructure rollback is not automatically safe for data. Accept a justified choice to fix forward, revert an application, or stop promotion based on the change's consequences.

Check version alignment across infrastructure, configuration, and job artifacts. The candidate should explain what is promoted between environments and how a failed version is prevented from reaching production.

Q5 - Short Python review and automation experience, 10 minutes

Open the candidate handout.

Format: entirely verbal. No editor, AWS SDK setup, full implementation, mocking framework, or written tests are required. There are two short exercises: four minutes for early return/shared defaults, two for Boolean selection, and four for actual automation experience. Typing is optional. The job records are an illustrative fixture, not a raw AWS API response. In exercise 1, each call must produce an independent result.

Exercise 1 - first 4 minutes:

> This function should return all failed job names. What does it actually return for the sample input? What change lets it examine every job? After making only that change, what do two successive calls to failed_jobs(jobs) return? Assume a fresh function definition and no other calls. Explain the second bug and its correction. After both fixes, what should happen for an empty list or when no jobs failed?

The input starts with a successful job, followed by two failed jobs. The current function is provided in the handout.

Expected reasoning - both bugs are required:

- `return` is inside the loop, so execution stops after the first record. With a fresh default list, the original sample returns `[]`; empty input returns `None`.
- Moving only `return failed` outside the loop leaves `failed=[]` shared between calls that omit the second argument. The first sample call returns `["load_orders", "load_products"]`; the second immediately returns

`["load_orders", "load_products", "load_orders", "load_products"]`. The default list is created when the function is defined and mutated by `append`. Even a later empty or all-successful input can return old failures.

- Remove the accumulator parameter and create `failed = []` inside each call. After both fixes, empty and all-successful inputs return `[]` independently of earlier calls. A `None` default with a fresh list per call is also acceptable.

The original early return hides accumulation for this sample because its first record succeeds. Ask the repeated-call question after the indentation correction. Both calls return the same list object before the second fix; a saved first result also changes when the second call appends. Mentioning aliasing is additional depth.

Interviewer reference only:

```
def failed_jobs(jobs):
    failed = []
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
    return failed
```

Exercise 2 - next 2 minutes: jobs to retry

Open exercise 2 in the handout. It selects `FAILED` or `TIMED_OUT` runs, but uses `if job["status"] == "FAILED" or "TIMED_OUT":`.

> What does retryable_jobs(runs) return, and what should it return? Why does this condition also select successful or cancelled jobs? Explain a small correction and which inputs you would check. No implementation is required.

Expected reasoning: the non-empty string `"TIMED_OUT"` is truthy; `or` does not implicitly repeat the comparison. Every input record passes the condition. The actual result is `["load_customers", "load_orders", "load_products", "load_prices"]`; the intended result is `["load_orders", "load_products"]`. Accept membership or an explicit comparison on each side of `or`.

```
def retryable_jobs(jobs):
    retryable = []
    for job in jobs:
        if job["status"] in ("FAILED", "TIMED_OUT"):
            retryable.append(job["name"])
    return retryable
```

Check mixed statuses, each included/excluded status, and empty input. Do not require a retry engine, SDK knowledge, or written tests. If the candidate struggles, ask what a non-empty string means in an `if`; record the prompting.

Remaining 4 minutes - discuss actual automation:

> Describe one Python automation you personally implemented. What did it do, how did you handle failures, and how did you check it before deployment?

Choose one follow-up:

- How did you make a failed or partially successful run visible to its caller or operator?
- Which cases did you check, and how did you isolate external services during testing?
- If an API call timed out, how would you decide whether retrying it was safe?
- What was one actual bug you found, and what evidence confirmed the fix?

Enough at regular/mid level: explains the early return and accumulation between calls in exercise 1, proposes independent results, and explains why the condition in exercise 2 selects every record and how to correct it. Gives a concrete account of automation and failure checking. Both exercises are included; the repeated-call question remains mandatory. Record any prompting needed for the shared default or Boolean condition. A list comprehension is acceptable but not required. If the issues are explained quickly, move on; do not fill the time with syntax trivia.

Additional depth: explains safe retries, structured failure reporting, or a meaningful distinction between unit and integration tests through their own work. Do not require implementing any of these during the interview.

Use both snippets as small code-reading checks. Identifying these bugs alone does not establish strong programming experience; use the project evidence to assess depth. Record the distinction between demonstrated code reading and self-reported implementation experience.

Q6 - Operations, networking, and incident communication, 11 minutes

Incident diagnosis - around 4 minutes:

> Return to your design from Q2, or assume S3 → SQS → Lambda → a reporting dataset. After a deployment, CI is green and new files arrive in S3, but the report has had no new data for 40 minutes. What are your first three checks? What evidence would confirm or reject your leading hypothesis?

Look for impact assessment, the last successful processing time, comparison with the latest change, and tracing a specific object or message through each stage. Useful evidence may include event delivery, queue depth and age, DLQ contents, Lambda invocations and errors, logs, writes, and the freshness of the consumed result. No Lambda errors may mean it is not being invoked at all.

Recovery and monitoring - around 3 minutes:

> The same message reaches Lambda twice. How would you avoid a duplicate business effect? What if one message in a batch fails? After fixing the issue, how would you recover the backlog and prove the data is complete?

SQS/Lambda processing must account for retries. Look for a stable operation key and durable tracking of its effect, or an idempotent write; an in-memory set is not sufficient. Partial batch responses, a DLQ, and controlled replay are useful options, without requiring the exact response JSON from memory. AWS: Lambda with SQS.

Ask one short monitoring follow-up:

> What alarm would detect stale or incomplete data even when every job is green, and who would own the response?

Look for a freshness/completeness measure tied to the agreed target, an owner, and a runbook. Recovery must account for data outcomes, not just restarting a service. Reverting a deployment does not necessarily recover lost data or remove duplicate processing effects. AWS: operational excellence for analytics.

AWS networking - around 2 minutes:

> After moving a Lambda function into private subnets, its API calls time out. How would you distinguish networking from IAM issues and check access to S3 and Secrets Manager?

Look for the destination service, routes, DNS, network rules, and endpoint or NAT configuration as appropriate. Putting Lambda in a public subnet does not automatically provide public internet access. Distinguish missing connectivity from an AccessDenied response; endpoint policies may also restrict access. Do not require a NAT gateway where an appropriate endpoint meets the requirement. AWS: VPC-connected Lambda connectivity.

Stakeholder update - final 2 minutes:

> Please give a brief status update to a business stakeholder about the incident. Explain the impact, what you know, what remains uncertain, what you are doing next, and when they can expect another update.

If time allows, replace a follow-up with:

> A stakeholder asks you to bypass the normal deployment process to restore service faster. How would you respond in a regulated environment?

Assess clarity, structure, facts versus hypotheses, and a realistic commitment to the next update. This is an incident-communication task within the technical interview, not a separate language test. For GxP, seek an account of the candidate's actual emergency-change process, testing, approvals, and evidence, rather than regulations recalled from memory.

Minutes 55-60 - Candidate questions and closing

Give the candidate the remaining time. Be ready to describe the actual AWS stack, team responsibilities, on-call expectations, review practices, autonomy, and onboarding. Record questions you cannot answer for follow-up. Interpret a lack of questions in the context of what has already been explained.

Closing script:

> We have covered the main technical topics. What would you like to know about the platform, the team, or the role?
>
> Thank you for walking me through your experience and the exercise. The next step is [insert the agreed recruitment process]. We will follow up through [insert the agreed contact channel].

Use only confirmed recruitment details when filling those placeholders.

4. Scorecard - complete after the interview

Assess the autonomy required for this particular team. This is a proposed rubric to agree with the team, not an official Capgemini hiring threshold.

Rating	Observable evidence
1	Cannot solve or explain the basic problem even after clarification
2	Solves the basics with prompts; needs support with deployment and failure handling
3	Independently solves a typical problem, tests the result, and identifies risks
4	Explains trade-offs, partial failures, and verification, supported by specific experience
N/O	Not observed: insufficient time or evidence; do not replace it with zero

Area	Suggested weight	Rating 1-4 / N/O	Evidence and support required
AWS data platform, IAM/KMS, and networking	20%		
Terraform / IaC and environment management	20%		
Data operations, troubleshooting, and recovery	20%		
CI/CD for infrastructure and processing jobs	15%		
Python code reading, automation, and testing approach	15%		
Ownership and communication in English	10%		

Use weights to organise evidence rather than deciding solely from an average. If the role requires autonomy, look for level 3 in AWS/IaC and data operations. Assess Python through the code review and specific project evidence. For level 2, record the support needed and whether the team can provide it. Obtain more evidence before making a recommendation if a critical area is N/O.

Assessing communication throughout

Observe whether the candidate can clarify requirements, explain a diagram and code, discuss trade-offs, understand follow-up questions, and provide a concise incident update in English. Assess job-related clarity and understanding, not accent or isolated grammar mistakes. Clarify ambiguous wording before concluding that an answer shows a technical gap; distinguish language difficulties from missing technical knowledge in your notes.

Debrief template

```
Confirmed personal contribution:

Python - demonstrated code reading, proposed correction, and project/testing evidence:

IaC - response to a partially failed deployment:

AWS data platform - data flow, access, and networking:

Data operations - freshness, replay, and verification of recovery:

Communication in English - observations across the interview:

Main area requiring support:

Unassessed areas:

Relevant experience - clarified responsibilities and recruiter follow-up:

Recommendation and evidence against this team's requirements:
```

5. Reserve questions - substitute, do not add time

Use these when the candidate introduces the topic or when you cannot run the exercise. Keep the total at 60 minutes. Do not replace all platform-design and operations questions with CV-specific details.

R8 - Exposed database password in GitHub, 2-4 minutes

Use when the candidate skips a topic or time remains. No coding is required.

> You discover that a database password was committed to a GitHub repository used by an AWS data pipeline. A teammate removes it in a new commit and says the problem is solved. Do you agree? What would you do next, and how would you check that the pipeline still works?

Two-minute baseline: removal is not enough. Revoke/rotate the exposed credential with the system owner, update consumers and test an authorised operation. History cleanup does not replace credential invalidation. GitHub: removing sensitive data.

If you have up to four minutes, choose one follow-up:

- What if the repository is private: does that change the need to act?
- The value in Secrets Manager was updated, but the job still fails. What would you check?
- What would you change to reduce the chance of this happening again?

Transition: "Let's switch to a short scenario about handling secrets."

Keep minutes 55-60 for candidate questions. Record the gap in any skipped topic: R8 does not establish that omitted skill. R8 is optional.

See the Polish scenario guide for R8 and R8 in the example answers.

Other reserve topics

- **CloudFormation to Terraform:** "How would you bring an existing resource under Terraform management without accidentally duplicating or deleting it?"

- **Infrastructure tests:** "Does a passing test run with administrator access prove that a consumer account has the intended permissions? How would you test a forbidden operation?"
- **S3 logging costs:** "Your CV mentions comparing logging architectures. What requirements, volumes, and costs did you compare? What was implemented, and what remained a recommendation?"
- **SQL reconciliation:** "Two systems report different record counts after a migration. How would you compare the datasets, including duplicates and missing identifiers?"
- **Containers:** "What practical problem have you solved with Docker, and how did you troubleshoot a failing container?" Expand only if relevant to the project.

Do not spend this hour on sorting algorithms, metaclasses, or definitions that do not help assess AWS data-platform work.

S3 migration - optional deeper follow-ups

> Existing data and new writes coexist during the migration. How did you verify that all required data reached the destination and could be read by consumers? What had to be true before cutover?

Choose a follow-up:

- What about objects created before replication was enabled, overwrites, and versions?
- Why does matching object counts not prove that the datasets are equivalent?
- How would you account for failed copies and retry them safely?

Look for separate handling of historical and ongoing data, an agreed migration scope, missing-object reports, and consumer-side tests. ETag is not always an MD5 of object data, so ask the candidate to justify the validation method. AWS: ETag and checksums.

Ordinary live replication is not an automatic backfill of pre-existing objects. Versioning and encryption configuration matter. AWS: replication scope, AWS: replication troubleshooting.

Probe proposals to grant AdministratorAccess without a diagnostic plan, or to consider a migration complete solely because an API call succeeded. First ask for clarification to distinguish shorthand from an intended action.

IAM access-key rotation - optional deeper follow-ups

> Why did this consumer need long-lived IAM access keys rather than a role and temporary credentials? Walk me through the rotation process. What happens if Lambda stops after creating a new key but before saving it to Secrets Manager?

Choose a follow-up:

- How does the next invocation recognise an in-progress rotation and avoid creating another key?
- What did the STS validation in your CV prove, and what did it not prove?
- When can you deactivate the old key, and what if a consumer caches the secret?

Look for a justified exception to temporary credentials, recorded process state, a rotation identifier, concurrency control, and keeping secrets out of logs. An IAM key's secret can only be retrieved at creation; a failure before storing it needs a recovery or orphan-cleanup strategy, not a promise to retrieve it again.

The rotation sequence includes creating the new key, updating and validating consumers, deactivating the old key, and deleting it later. The two-key limit also includes inactive keys. A successful `GetCallerIdentity` confirms identity, not access to the intended S3 data or database. AWS: GetCallerIdentity, AWS: access keys, AWS: key rotation.

In a short follow-up, seek awareness of partial failures, retries, and consumer impact rather than a fully specified rotation protocol.